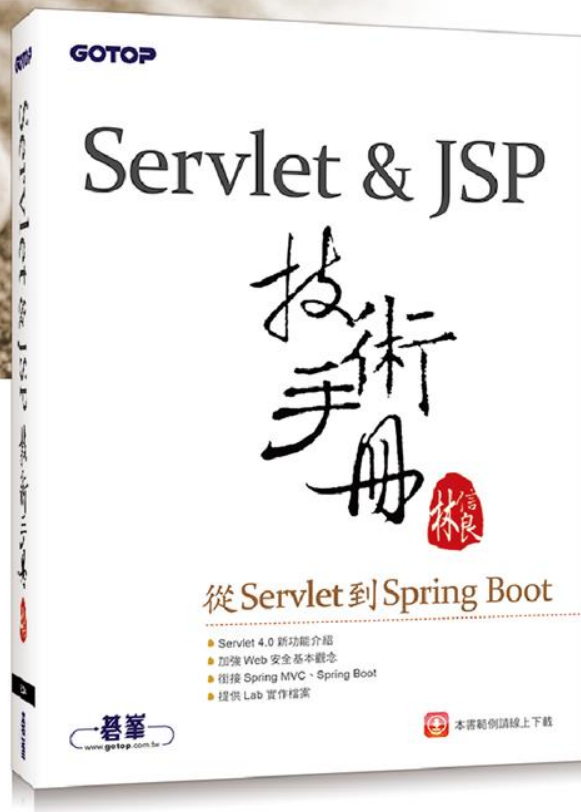




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

CHAPTER 3

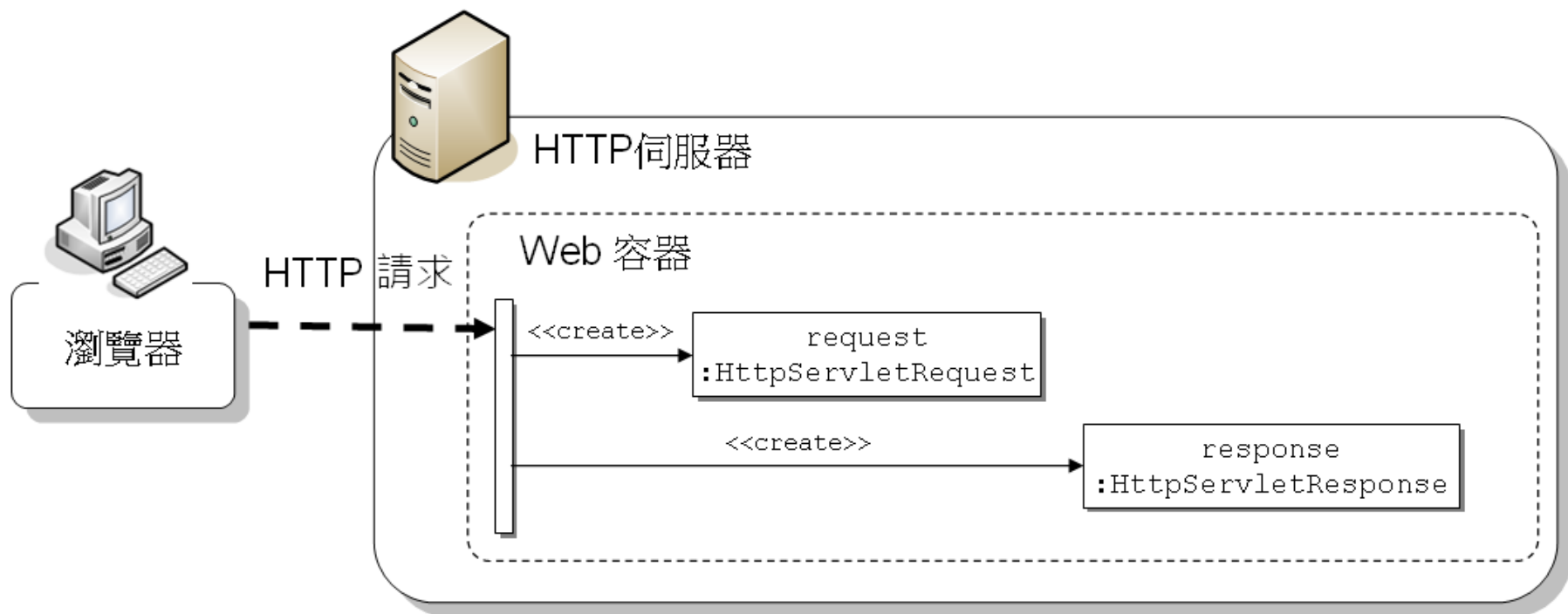
• 請求與回應



學習目標

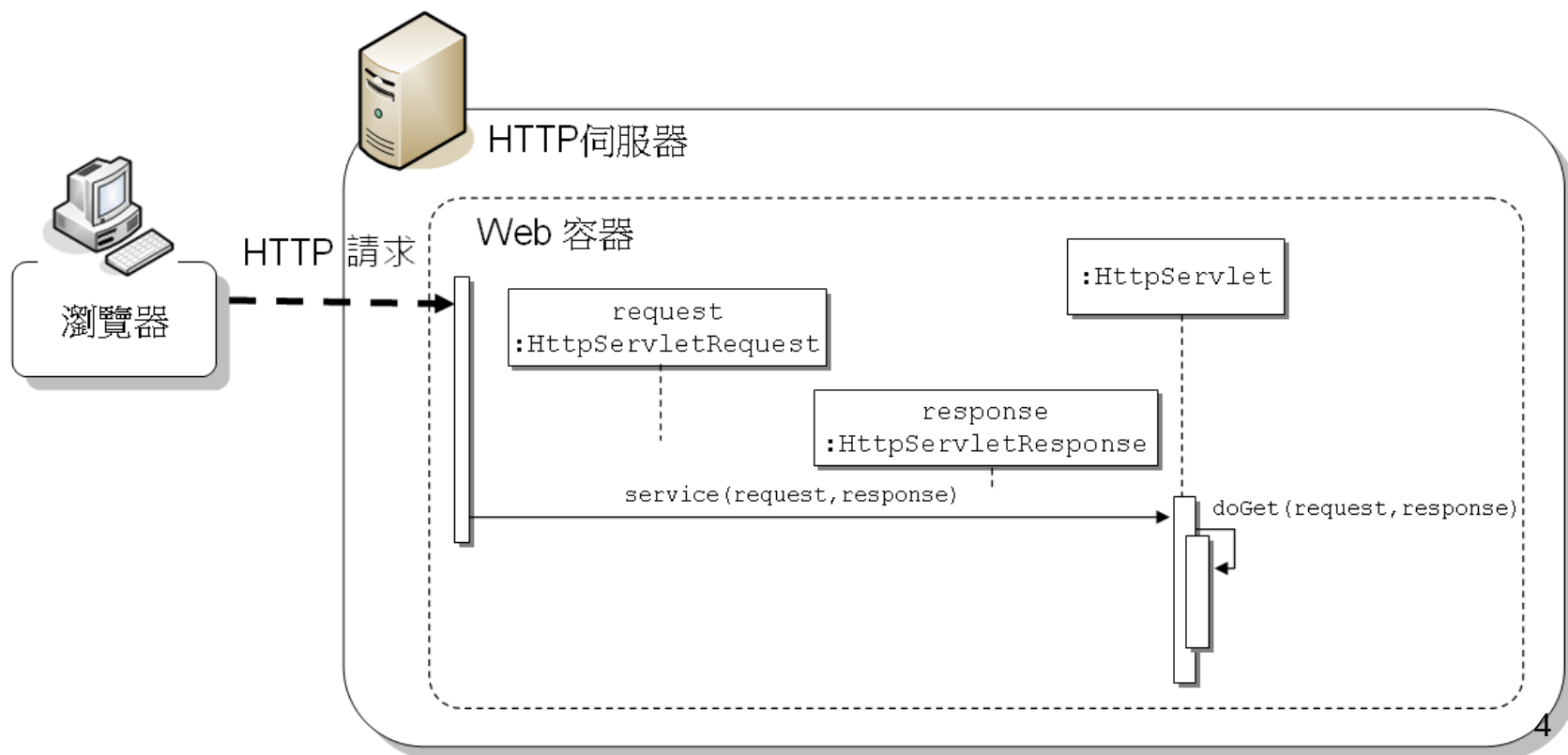
- 取得請求參數與標頭
- 處理中文字元請求與回應
- 設定與取得請求範圍屬性
- 使用轉發、包含、重新導向

Web容器作了什麼？

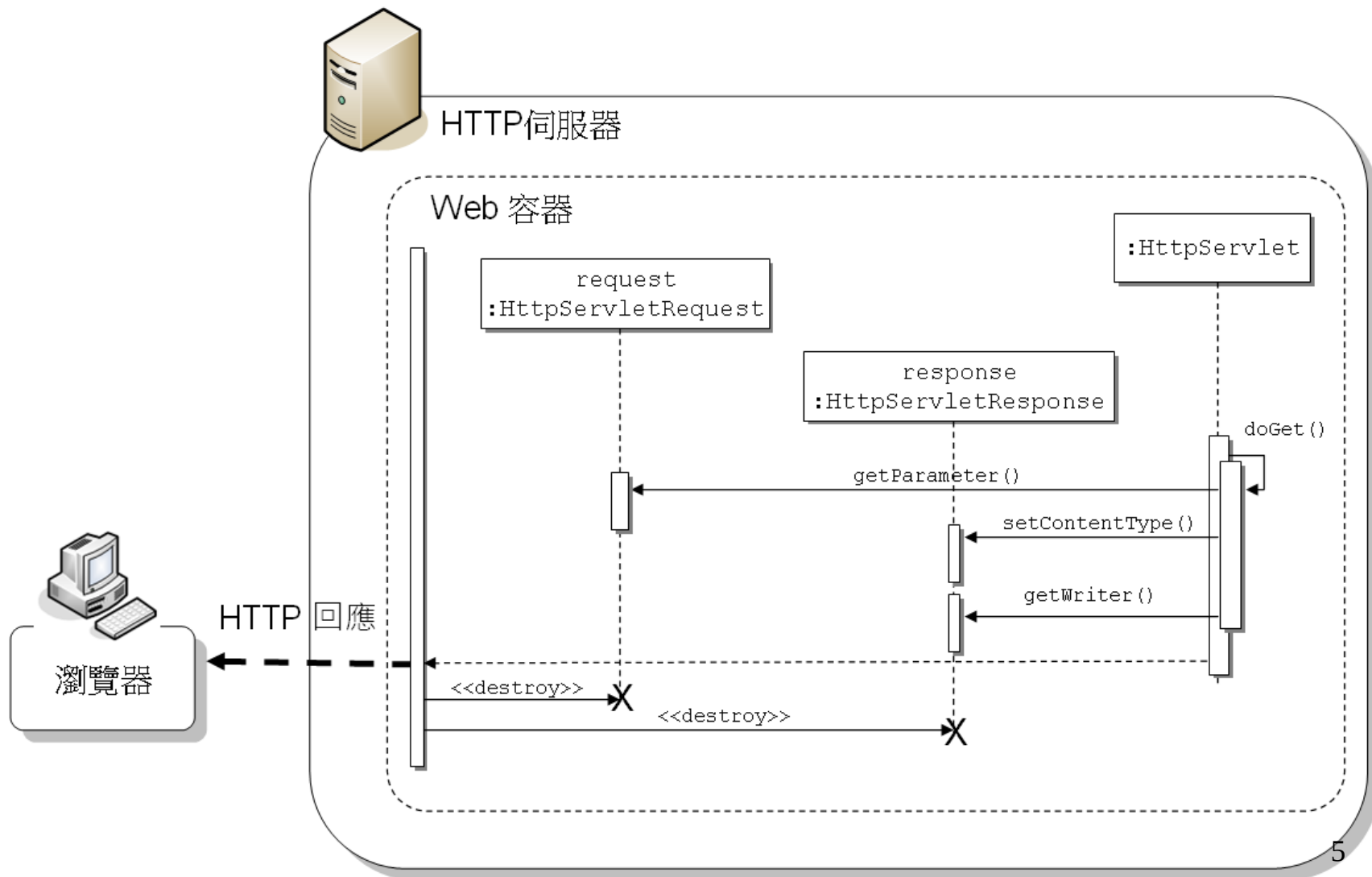


Web容器作了什麼？

- `HttpServletRequest`、`HttpServletResponse` 都是介面



Web容器作了什麼？

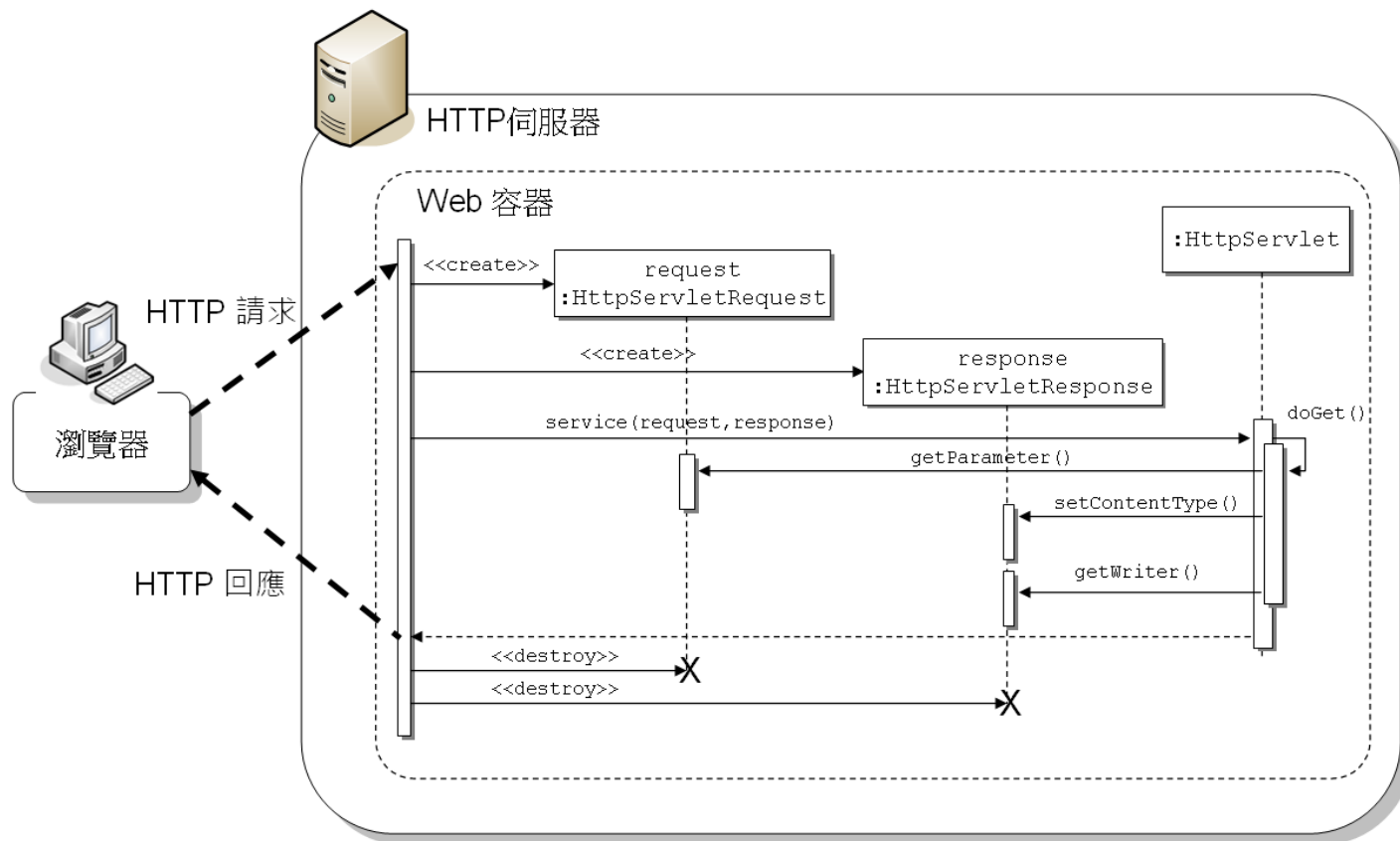


Web容器作了什麼？

- 請求資訊的收集
- 建立`HttpServletRequest`物件
- 建立`HttpServletResponse`物件
- 輸出HTTP回應之轉換
- 銷毀、回收`HttpServletRequest`物件
- 銷毀、回收`HttpServletResponse`物件
- ...

Web容器作了什麼？

- 必須了解Web容器管理物件生命週期的方式，否則就會引來不必要的錯誤



doXXX()方法？

- service() 方法簽署

```
public void service(ServletRequest req, ServletResponse res)  
    throws ServletException, IOException;
```

- 請求/回應物件的基本行為是規範在 `ServletRequest`、`ServletResponse`（套件是 `javax.servlet`）
- 與HTTP相關的行為，則分別由兩者的子介面 `HttpServletRequest`、`HttpServletResponse`（套件是 `javax.servlet.http`）定義

doXXX()方法？

- Web容器建立HttpServletRequest、HttpServletResponse的實作物件

```
public void service(ServletRequest req, ServletResponse res)
    throws ServletException, IOException {
    HttpServletRequest request;
    HttpServletResponse response;

    try {
        request = (HttpServletRequest) req;
        response = (HttpServletResponse) res;
    } catch (ClassCastException e) {
        throw new ServletException("non-HTTP request or response");
    }
    service(request, response);
}
```

另一個版本的**service()**方法

doXXX()方法？

```
protected void service(HttpServletRequest req,
                        HttpServletResponse resp)
    throws ServletException, IOException {
    String method = req.getMethod();
    if (method.equals(METHOD_GET)) {
        long lastModified = getLastModified(req);
        if (lastModified == -1) {
            doGet(req, resp);
        } else {
            long ifModifiedSince =
                req.getDateHeader(HEADER_IFMODSINCE);
            if (ifModifiedSince < (lastModified / 1000 * 1000)) {
                maybeSetLastModified(resp, lastModified);
                doGet(req, resp);
            } else {
                resp.setStatus(HttpServletResponse.SC_NOT_MODIFIED);
            }
        }
    }
}
```

doXXX()方法？

```
} else if (method.equals(METHOD_HEAD)) {  
    long lastModified = getLastModified(req);  
    maybeSetLastModified(resp, lastModified);  
    doHead(req, resp);  
} else if (method.equals(METHOD_POST)) {  
    doPost(req, resp);  
} else if (method.equals(METHOD_PUT)) {  
    略...  
}
```

doXXX()方法？

- doGet () 處理 HTTP GET 請求
- doPost () 處理 HTTP POST 請求
- doPut () 處理 HTTP PUT 請求
- delete () 處理 HTTP DELETE 請求
- doHead () 處理 HTTP HEAD 請求
- doOptions () 處理 HTTP OPTIONS 請求
- doTrace () 處理 HTTP TRACE 請求

doXXX()方法？

- 如果客戶端發出了沒有實作的請求會如何？

```
protected void doGet(HttpServletRequest req,
                      HttpServletResponse resp)
    throws ServletException, IOException {
    String protocol = req.getProtocol();
    String msg =
        lStrings.getString("http.method_get_not_supported");
    if (protocol.endsWith("1.1")) {
        resp.sendError(
            HttpServletResponse.SC_METHOD_NOT_ALLOWED, msg);
    } else {
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST, msg);
    }
}
```

doXXX()方法？

HTTP Status 405 – Method Not Allowed

Type Status Report

Message HTTP method GET is not supported by this URL

Description The method received in the request-line is known by the origin server but not supported by the target resource.

Apache Tomcat/9.0.2

doXXX()方法？

- 可以實作`getLastModified()`方法決定是否呼叫`doGet()`方法
- 在GET與POST都需要相同處理的情境

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    processRequest(req, resp);
}

protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    processRequest(req, resp);
}

protected void processRequest(HttpServletRequest req,
                                HttpServletResponse resp)
    throws ServletException, IOException {
    // 處理請求...
}
```

處理請求參數

- `getParameter()`
- `getParameterValues()`
- `getParameterNames()`
- `getParameterMap()`

處理請求參數

- 永遠別假設使用者會按照你的期望提供請求

name=%3Csmall%3EJustin%3C/small%3E



Hello! Justin!

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello</title>
  </head>
  <body>
    <h1> Hello! <small>Justin</small>!</h1>
  </body>
</html>
```

處理請求參數

- 未經過濾的請求參數值形成Web網站的弱點
- 引發各種可能注入（**Injection**）攻擊可能性

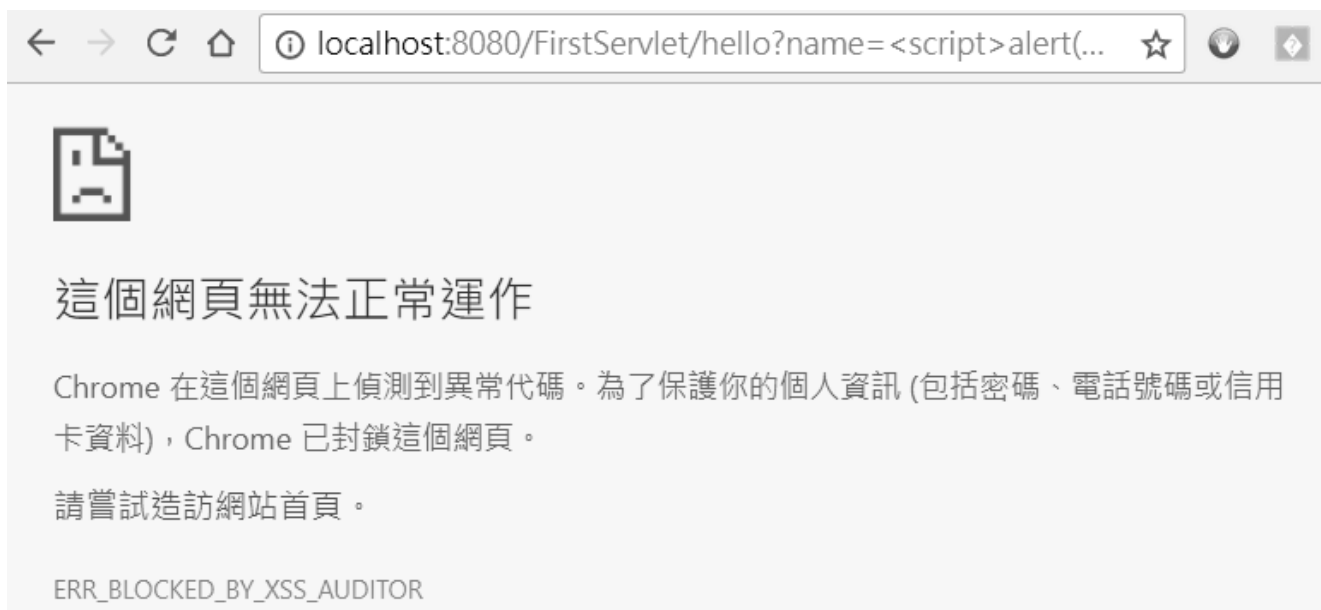
`name=%3Cscript%3Ealert(%27Attack%27)%3C/script%3E`

`name=<script>alert('Attack')</script>`



處理請求參數

- 現代瀏覽器能防範 基本的XSS模式



處理請求參數

① 使用 `Optional`

```
String name = Optional.ofNullable(request.getParameter("name"))  
    .map(value -> value.replaceAll("<", "&lt;"))  
    .map(value -> value.replaceAll(">", "&gt;")) ② 取代為 HTML 實體名稱  
    .orElse("Guest"); ③ 沒有提供請求參數時的預設值
```

 http://localhost:8080/Request/hello?name=%3Cscript%3Ealert(%27Attack%27)%3C/script%3E

Hello! <script>alert('Attack')</script>!

處理請求標頭

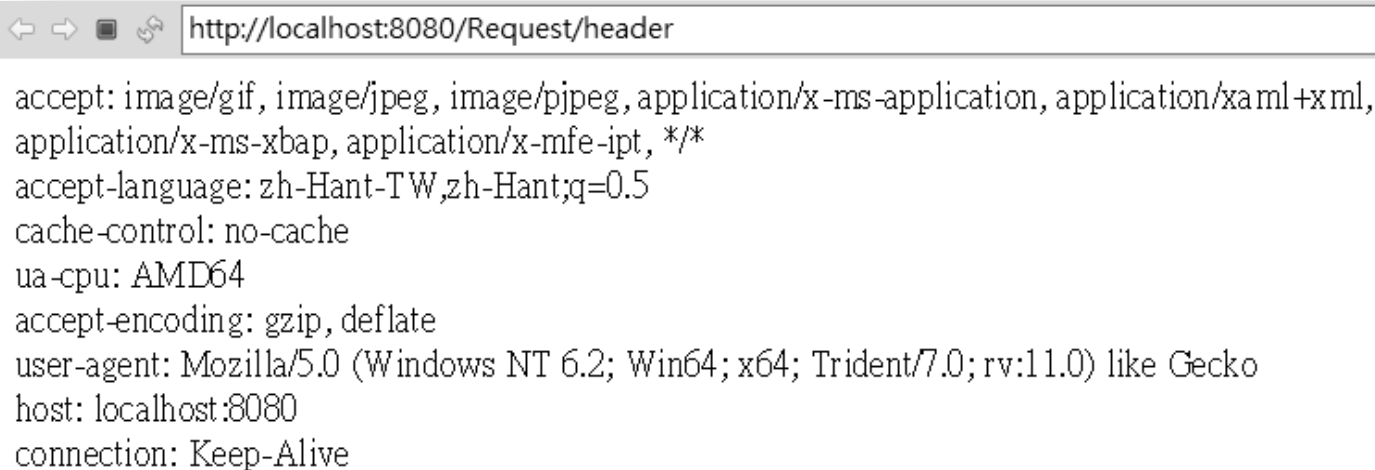
- `getHeader()`
- `getHeaders()`
- `getHeaderNames()`

處理請求標頭

```
Collections.list(request.getHeaderNames()) ← ❶ 取得全部標頭名稱
    .forEach(name -> {
        out.printf("%s: %s<br>",
            name, request.getHeader(name));
    });

out.print("</body>");
out.print("</html>");
```

↑
❷ 取得標頭值



http://localhost:8080/Request/header

```
accept: image/gif, image/jpeg, image/pjpeg, application/x-ms-application, application/xhtml+xml,
application/x-ms-xbap, application/x-mfe-ipt, */*
accept-language: zh-Hant-TW,zh-Hant;q=0.5
cache-control: no-cache
ua-cpu: AMD64
accept-encoding: gzip, deflate
user-agent: Mozilla/5.0 (Windows NT 6.2; Win64; x64; Trident/7.0; rv:11.0) like Gecko
host: localhost:8080
connection: Keep-Alive
```

瀏覽器用UTF-8發送請求

- POST請求參數編碼處理

```
req.setCharacterEncoding("UTF-8");
```

- GET請求參數編碼處理

- Tomcat 8.0 以後預設URI編碼為UTF-8
- Tomcat 8.0 之前預設URI編碼為ISO-8859-1

```
String name = req.getParameter("name");  
String name = new String(name.getBytes("ISO-8859-1"), "UTF-8");
```

POST請求參數編碼處理

- 沒有Content-Type標頭，
getCharacterEncoding() 傳回null
- 容器若使用ISO-8859-1，客戶端使用UTF-8發送非ASCII字元的請求參數
- 使用getParameter() 等方法會取得亂碼

POST請求參數編碼處理

- 網頁編碼UTF-8，表單使用POST發出「林」
– 林 → %E6%9E%97

- 瀏覽器相當於作了這個動作

```
String text = java.net.URLEncoder.encode("林", "UTF-8");
```

- 容器若使用ISO-8859-1來處理編碼

```
String text = java.net.URLDecoder.decode("%E6%9E%97", "ISO-8859-1");
```

POST請求參數編碼處理

- 使用`HttpServletRequest`的 **`setCharacterEncoding()`** 方法
- 在取得任何請求值之「前」

```
req.setCharacterEncoding("UTF-8");
```

- 相當於要求容器作這個動作

```
String text = java.net.URLDecoder.decode("%E6%9E%97", "UTF-8");
```

POST請求參數編碼處理

- 從Servlet 4.0開始，可以在web.xml中加入
<request-character-encoding>

`<request-character-encoding>UTF-8</request-character-encoding>`

GET請求參數編碼處理

- `setCharacterEncoding()` 只對POST產生作用
 - Overrides the name of the character encoding used in the body of this request.
 - 請求用GET發送時，沒有定義是否影響Web容器處理編碼的方式
 - Tomcat在GET時，`setCharacterEncoding()` 方法設定編碼就不會有作用

GET請求參數編碼處理

- 若瀏覽器使用UTF-8處理字元，Web容器預設使用ISO-8859-1作為URI編碼（Tomcat 8.0之前的版本）

```
String name = req.getParameter("name");  
String name = new String(name.getBytes("ISO-8859-1"), "UTF-8");
```

GET請求參數編碼處理

- 若瀏覽器使用UTF-8處理字元，相當於作了這個動作

```
String text = java.net.URLEncoder.encode("林", "UTF-8");
```

- Web容器預設使用ISO-8859-1作為URI編碼

```
String text = java.net.URLDecoder.decode("%E6%9E%97", "ISO-8859-1");
```

- 用以下編碼轉換來得到正確的「林」字元

```
text = new String(name.getBytes("ISO-8859-1"), "UTF-8");
```

讀取請求本體

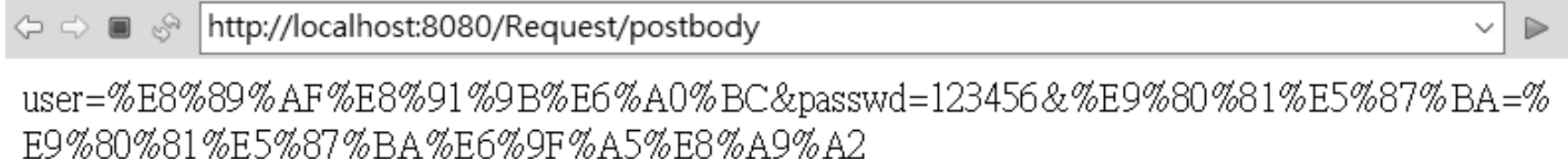
- `getReader()` 傳回 `BufferedReader`
- `getInputStream()` 傳回 `ServletInputStream`

```
protected void doPost(  
    HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
    PrintWriter out = response.getWriter();  
    out.println("<!DOCTYPE html>");  
    out.println("<html>");  
    out.println("<body>");  
    out.println(bodyContent(request.getReader())); ← 取得 BufferedReader  
    out.println("</body>");  
    out.println("</html>");  
}  
  
private String bodyContent(BufferedReader reader) throws IOException {  
    String input = null;  
    StringBuilder requestBody = new StringBuilder();  
    while((input = reader.readLine()) != null) {  
        requestBody.append(input)  
            .append("<br>");  
    }  
    return requestBody.toString();  
}
```

getReader()

- UTF-8網頁

```
<form action="postbody" method="post">  
  Username: <input type="text" name="user"><br>  
  Password: <input type="password" name="passwd"><br>  
  <input type="submit" name="送出查詢">  
</form>
```



user=%E8%89%AF%E8%91%9B%E6%A0%BC&passwd=123456&%E9%80%81%E5%87%BA=%E9%80%81%E5%87%BA%E6%9F%A5%E8%A9%A2

<form>標籤的**enctype**屬性

- 預設值"application/x-www-form-urlencoded"
- 上傳檔案，要設為"multipart/form-data"

getReader()

• 上傳檔案

```
<form action="body.view" method="post"
      enctype="multipart/form-data">
  選擇檔案: <input type="file" name="filename" value="" /><br>
  <input type="submit" value="Upload" name="upload" />
</form>
```

```
-----195582364627982
Content-Disposition: form-data; name="filename"; filename="caterpillar.jpg"
Content-Type: image/jpeg

ÿøÿà JFIF  HHÿâ øICC_PROFILE  è mntrRGB XYZ Û  $-acsp öÖ Ó-)ø=P`òU@xBúäÊf9
desc DybXYZ À bTRC Ô  dmdd à^gXYZ
```

總之是一堆奇奇怪怪的字元，這些字元是實際的檔案內容。

```
-----195582364627982
Content-Disposition: form-data; name="upload"

Upload
-----195582364627982-
```

getInputStream()

- 要取得上傳的檔案，基本方式就是判斷檔案的開始與結束區段
- 使用**getInputStream()**取得
ServletInputStream
 - InputStream子類別，代表請求本體串流物件

同一個請求期間

- `getReader()` 與 `getInputStream()` 只能擇一呼叫
- 若同一請求期間兩者都有呼叫，則會丟出 `IllegalStateException` 例外

getPart () 、 getParts ()

- 在Servlet 3.0中，新增了**Part**介面
- 可以透過HttpServletRequest的**getPart ()**取得Part實作物件

```
@MultipartConfig      ← ❶ 必須設定此標註才能使用 getPart() 相關 API
@WebServlet("/photo")
public class Photo extends HttpServlet {
    private final Pattern fileNameRegex =
        Pattern.compile("filename=\"(.*)\"");

    @Override
    protected void doPost(
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        Part photo = request.getPart("photo"); ← ❷ 使用 getPart() 取得 Part 物件
        String filename = getSubmittedFileName(photo);
        write(photo, filename);
    }

    private String getSubmittedFileName(Part part) { ← ❸ 取得上傳檔名
        String header = part.getHeader("Content-Disposition");
        Matcher matcher = fileNameRegex.matcher(header);
        matcher.find();

        String filename = matcher.group(1);
        if(filename.contains("\\\"")) {
            return filename.substring(filename.lastIndexOf("\\\"") + 1);
        }
        return filename;
    }
    ↓ ❹ 儲存檔案
    private void write(Part photo, String filename)
        throws IOException, FileNotFoundException {
```

```
}
```

④ 儲存檔案
↓

```
private void write(Part photo, String filename)
    throws IOException, FileNotFoundException {
    try(InputStream in = photo.getInputStream();
        OutputStream out = new FileOutputStream(
            String.format("c:/workspace/%s", filename))) {
        byte[] buffer = new byte[1024];
        int length = -1;
        while ((length = in.read(buffer)) != -1) {
            out.write(buffer, 0, length);
        }
    }
}
```

@MultipartConfig

- `fileSizeThreshold`：上傳檔案大小超過設定門檻的話，會先寫入暫存檔案
- `location`：寫入檔案時的目錄
- `maxFileSize`：限制上傳檔案大小
- `maxRequestSize`：限制multipart/form-data 請求個數

@WebServlet("/photo2")

public class Photo2 extends HttpServlet {

private final Pattern fileNameRegex =

Pattern.compile("filename=\"(.*)\"");

@Override

protected void doPost(

HttpServletRequest request, HttpServletResponse response)

throws ServletException, IOException {

request.setCharacterEncoding("UTF-8"); ← ❷ 為了處理中文檔名

Part photo = request.getPart("photo");

String filename = getSubmittedFileName(photo);

photo.write(filename); ← ❸ 將檔案寫入 location 指定的目錄

}

private String getSubmittedFileName(Part part) {

String header = part.getHeader("Content-Disposition");

Matcher matcher = fileNameRegex.matcher(header);

matcher.find();

String filename = matcher.group(1);

if(filename.contains("\\\")) {

return filename.substring(filename.lastIndexOf("\\\"") + 1);

}

return filename;

}

}

多個檔案要上傳

- 可以使用 `getParts()` 方法
- 傳回一個 `Collection<Part>`，當中是每個上傳檔案的 `Part` 物件

```
protected void doPost(  
    HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {
```

```
    request.setCharacterEncoding("UTF-8");
```

```
    request.getParts()
```

```
        .stream() ← ❶ 使用 Stream
```

```
        .filter(part -> part.getName().startsWith("file"))
```

```
        .forEach(this::write);
```

```
}
```

❷ 只處理上傳檔案區段



使用web.xml設定

```
...  
<servlet>  
  <servlet-name>UploadServlet</servlet-name>  
  <servlet-class>cc.openhome.UploadServlet</servlet-class>  
  <multipart-config>  
    <location>c:/workspace</location>  
  </multipart-config>  
</servlet>  
...
```

RequestDispatcher

- 使用 `HttpServletRequest` 的 **`getRequestDispatcher()`** 方法取得

```
RequestDispatcher dispatcher =  
    request.getRequestDispatcher("some.do");
```

<<interface>> RequestDispatcher
+ <i>forward(request : ServletRequest, response : ServletResponse) : void</i> + <i>include(request : ServletRequest, response : ServletResponse) : void</i>

使用**include()** 方法

- 將另一個Servlet執行流程包括至目前Servlet執行流程

```
@Override
protected void doGet(HttpServletRequest req,
                      HttpServletResponse resp)
    throws ServletException, IOException {
    PrintWriter out = resp.getWriter();
    out.println("Some do one...");
    RequestDispatcher dispatcher =
        req.getRequestDispatcher("other.view");
    dispatcher.include(req, resp);
    out.println("Some do two...");
    out.close();
}
```

使用**include()** 方法

- 取得RequestDispatcher時，也可以包括查詢字串

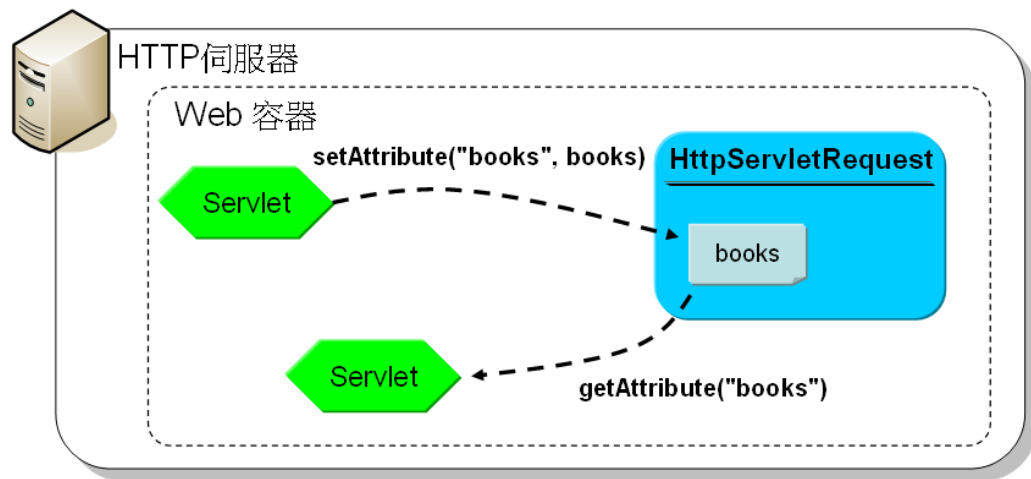
```
req.getRequestDispatcher("other.view?data=123456")  
    .include(req, resp);
```

請求範圍屬性

- `HttpServletRequest` 上與請求範圍屬性有關的幾個方法：
 - `setAttribute()`：指定名稱與物件設定屬性
 - `getAttribute()`：指定名稱取得屬性
 - `getAttributeNames()`：取得所有屬性名稱
 - `removeAttribute()`：指定名稱移除屬性

請求範圍屬性

```
...  
List<Book> books = bookDAO.query("ServletJSP");  
request.setAttribute("books", books);  
request.getRequestDispatcher("result.view")  
    .include(request, response);  
...
```



```
...  
List<Book> books = (List<Book>) request.getAttribute("books");  
...
```

java. 或 javax. 開頭的名稱

- `javax.servlet.include.request_uri`
- `javax.servlet.include.context_path`
- `javax.servlet.include.servlet_path`
- `javax.servlet.include.path_info`
- `javax.servlet.include.query_string`
- `javax.servlet.include.mapping` (Servlet 4.0 新增)

java. 或 javax. 開頭的名稱

- `RequestDispatcher.INCLUDE_REQUEST_URI`
- `RequestDispatcher.INCLUDE_CONTEXT_PATH`
- `RequestDispatcher.INCLUDE_SERVLET_PATH`
- `RequestDispatcher.INCLUDE_PATH_INFO`
- `RequestDispatcher.INCLUDE_QUERY_STRING`
- `RequestDispatcher.INCLUDE_MAPPING` (Servlet 4.0 新增)

使用forward() 方法

- 要將請求處理轉發給別的Servlet
- 對客戶端的回應同時也轉發給另一個Servlet
- 目前的Servlet不能有任何回應確認，否則會丟出IllegalStateException

java. 或 javax. 開頭的名稱

- `javax.servlet.forward.request_uri`
- `javax.servlet.forward.context_path`
- `javax.servlet.forward.servlet_path`
- `javax.servlet.forward.path_info`
- `javax.servlet.forward.query_string`
- `javax.servlet.forward.mapping` (Servlet 4.0 新增)

java. 或 javax. 開頭的名稱

- `RequestDispatcher.FORWARD_REQUEST_URI`
- `RequestDispatcher.FORWARD_CONTEXT_PATH`
- `RequestDispatcher.FORWARD_SERVLET_PATH`
- `RequestDispatcher.FORWARD_PATH_INFO`
- `RequestDispatcher.FORWARD_QUERY_STRING`
- `RequestDispatcher.FORWARD_MAPPING` (Servlet 4.0 新增)

Model 2 / Controller

```
@WebServlet("/hello")
public class HelloController extends HttpServlet {
    private HelloModel model = new HelloModel();
    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {
        String name = request.getParameter("user"); ← ❶ 收集請求參數
        String message = model.doHello(name); ← ❷ 委託 HelloModel 物件處理
        request.setAttribute("message", message); ← ❸ 將結果訊息設定至請求物件
        request.getRequestDispatcher("hello.view") ← ❹ 轉發給 hello.view 進行回應
            .forward(request, response);
    }
}
```

Model 2 / Model

```
public class HelloModel {  
    private Map<String, String> messages = new HashMap<>();  
  
    public HelloModel() {  
        messages.put("caterpillar", "Hello");  
        messages.put("Justin", "Welcome");  
        messages.put("momor", "Hi");  
    }  
  
    public String doHello(String user) {  
        String message = messages.get(user);  
        return String.format("%s, %s!", message, user);  
    }  
}
```

```
@WebServlet("/hello.view")
public class HelloView extends HttpServlet {
    private String htmlTemplate =
        "<!DOCTYPE html>"
        + "<html>"
        + "  <head>"
        + "    <meta charset='UTF-8'>"
        + "    <title>%s</title>"
        + "  </head>"
        + "  <body>"
        + "    <h1>%s</h1>"
        + "  </body>"
        + "</html>";

    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {
        String user = request.getParameter("user"); ← ❶ 取得請求參數
        String message =
            (String) request.getAttribute("message"); ← ❷ 取得請求屬性
        String html =
            String.format(htmlTemplate, user, message); ← ❸ 產生 HTML 結果
        response.getWriter().print(html); ← ❹ 輸出 HTML 結果
    }
}
```

Model 2 / View

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>${param.user}</title> ← 利用 Expression Language 取得 user 請求參數
  </head>
  <body>
    <h1>${message}</h1> ← 利用 Expression Language 取得請求範圍中設定的屬性值
  </body>
</html>
```

HttpServletResponse

- `setHeader()`
- `addHeader()`
- `setIntHeader()`
- `addIntHeader()`
- `setDateHeader()`
- `addDateHeader()`

HttpServletResponse

- 在回應確認之後設定的標頭，會被容器忽略
- 與緩衝區相關
 - `getBufferSize()`
 - `setBufferSize()`
 - `isCommitted()`
 - `reset()`
 - `resetBuffer()`
 - `flushBuffer()`

HttpServletResponse

- 在呼叫HttpServletResponse的
getWriter()或 getOutputStream() 方法之後呼叫setBufferSize()，會丟出
IllegalStateException
- 在回應已確認後呼叫reset()、
resetBuffer()會丟出
IllegalStateException

HttpServletResponse

- 若被容器關閉，則必須出清所有的回應內容
 - Servlet的`service()`方法已結束
 - 回應的內容長度超過`HttpServletResponse`的`setContentLength()`所設定的長度
 - 呼叫了`sendRedirect()`方法
 - 呼叫了`sendError()`方法
 - 呼叫了`AsyncContext`的`complete()`方法

HttpServletResponse

- 使用 `getWriter()` 取得 `PrintWriter` 物件
- 字元編碼預設是 `ISO-8859-1`

設定Locale

- 瀏覽器如果有發送Accept-Language標頭
- 可以使用HttpServletRequest的 **getLocale()** 來取得一個**Locale**物件
- 代表客戶端可接受的語系
- 可以使用HttpServletResponse的 **setLocale()** 來設定地區（Locale）資訊
- **setLocale()** 也會設定HTTP回應的Content-Language標頭

設定Locale

- 將HTTP回應的Content-Language設定為zh-TW，而字元編碼處理設定為BIG5

```
resp.setLocale(Locale.TAIWAN);
```

- 在web.xml中設定預設的區域與編碼對應

```
<locale-encoding-mapping-list>  
  <locale-encoding-mapping>  
    <locale>zh_TW</locale>  
    <encoding>UTF-8</encoding>  
  </locale-encoding-mapping>  
</locale-encoding-mapping-list>
```

設定字元編碼

- 呼叫HttpServletResponse的
setCharacgerEncoding()

```
resp.setCharacterEncoding("UTF-8");
```

- 使用HttpServletResponse的
setContentType()時，指定charset

```
resp.setContentType("text/html; charset=UTF-8");
```

設定字元編碼

- 如果使用了 `setCharacterEncoding()` 或 `setContentType()` 時指定了 `charset`，則 `setLocale()` 就會被忽略

```
protected void doPost(  
    HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {  
    request.setCharacterEncoding("UTF-8"); ← ❶ 設定請求物件字元編碼  
    response.setContentType("text/html; charset=UTF-8"); ← ❷ 設定內容類型  
  
    PrintWriter out = response.getWriter(); ← ❸ 取得輸出物件  
    out.println("<!DOCTYPE html>");  
    out.println("<html>");  
    out.println("<body>");  
  
    out.printf("聯絡人:<a href='mailto:%s'>%s</a>%n",  
        request.getParameter("email"),  
        request.getParameter("user") ← ❹ 取得請求參數值  
    );  
  
    out.println("<br>喜愛的寵物類型");  
    out.println("<ul>");  
  
    Arrays.asList(request.getParameterValues("type")) ← ❺ 取得複選項  
        .forEach(type -> out.printf("<li>%s</li>%n", type)); ← 請求參數值  
  
    out.println("</ul>");  
    out.println("</body>");  
    out.println("</html>");  
}
```

getOutputStream()

- 取得**ServletOutputStream**實例，為OutputStream的子類別

```
protected void doPost(  
    HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {  
    String passwd = request.getParameter("passwd");  
    if ("123456".equals(passwd)) {  
        response.setContentType("application/pdf"); ← ❶ 設定內容類型  
  
        try(InputStream in =  
            getServletContext().getResourceAsStream("/WEB-INF/jdbc.pdf");  
            OutputStream out = response.getOutputStream()) {  
                byte[] buffer = new byte[1024];  
                int length = -1;  
                while ((length = in.read(buffer)) != -1) { ← ❷ 取得輸入串流  
                    out.write(buffer, 0, length);  
                }  
            }  
        }  
    }  
}
```

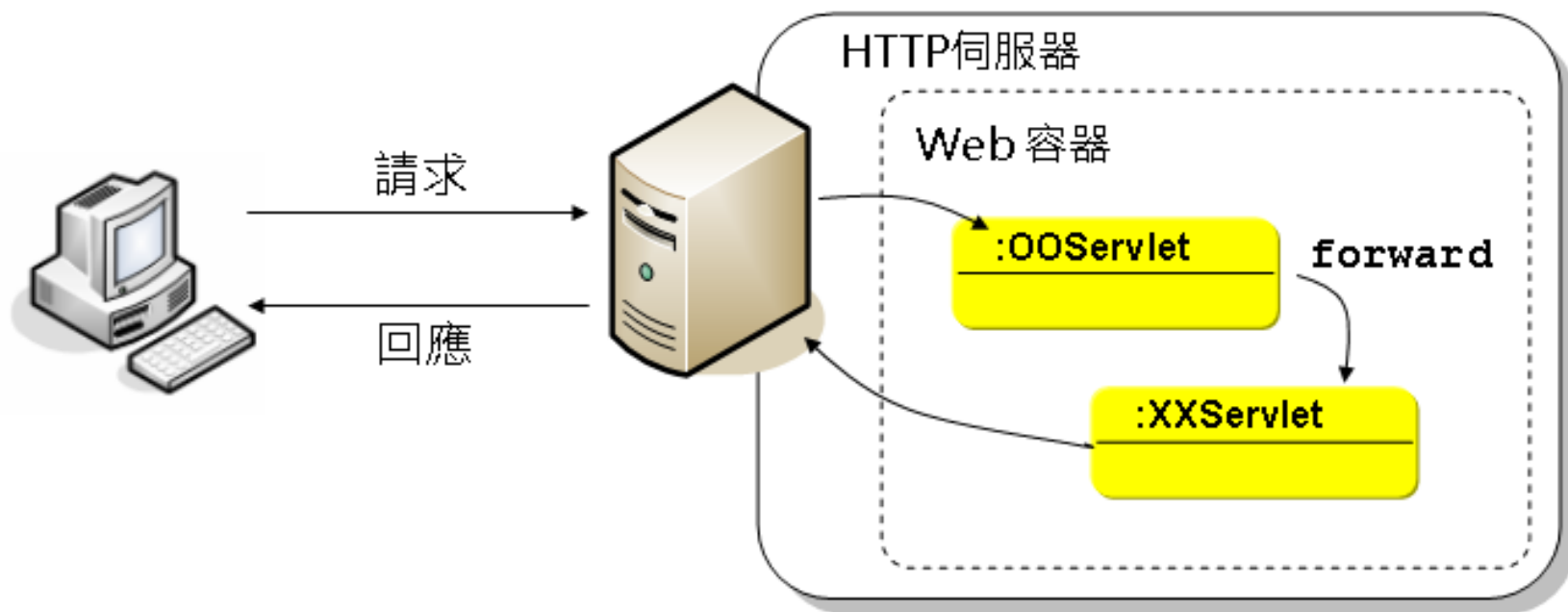
↓

↑

❸ 取得輸出串流

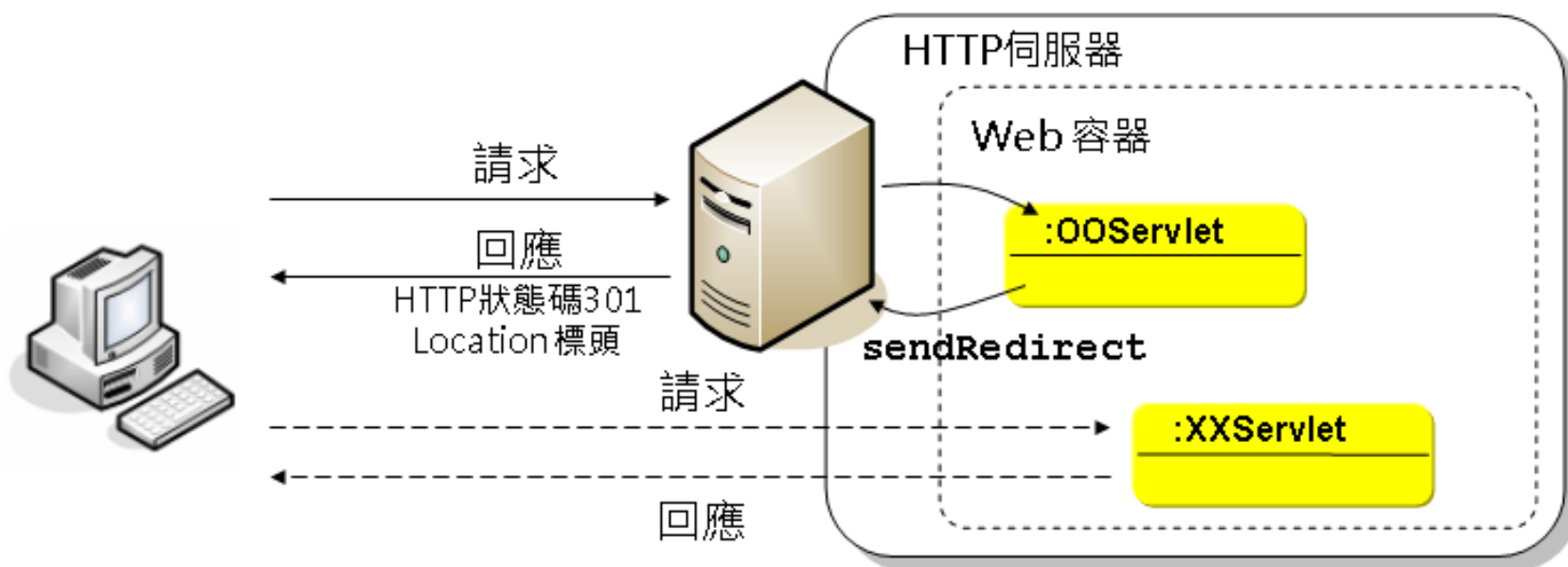
❹ 讀取 PDF 並輸出

forward() 方法



sendRedirect()

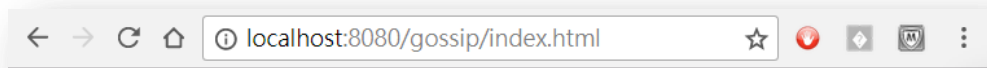
```
response.sendRedirect("https://openhome.cc");
```



sendError()

```
protected void doGet(HttpServletRequest req,
                    HttpServletResponse resp)
                    throws ServletException, IOException {
    String protocol = req.getProtocol();
    String msg = Strings.getString("http.method_get_not_supported");
    if (protocol.endsWith("1.1")) {
        resp.sendError(
            HttpServletResponse.SC_METHOD_NOT_ALLOWED, msg);
    } else {
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST, msg);
    }
}
```

微網誌



Gossip ... XD

- 談天說地不奇怪
- 分享訊息也可以
- 隨意寫寫表心情

還不是會員?

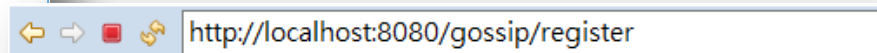
會員登入

名稱:

密碼:

登入

[忘記密碼?](#)

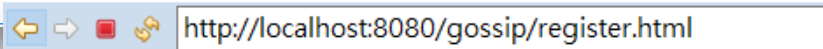


表單填寫錯誤

表單填寫錯誤

- 未填寫郵件或格式不正確
- 未填寫使用者名稱或格式不正確
- 請確認密碼符合格式並再度確認密碼

[返回註冊表單](#)



會員申請

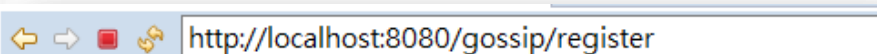
郵件位址:

名稱 (最大 16 字元):

密碼 (8 到 16 字元):

確認密碼:

註冊



成功送出表單

caterpillar 成功送出表單

[回首頁](#)